

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №1. Инструмент сборки Maven

Выполнил: студент группы БПИ2401

Исламов Эмин Маратович

Проверил: Харрасов Камиль Раисович

Цель работы: Освоить Apache Maven как инструмент автоматизации сборки.

Ход работы

1) Создание базового Maven-проекта и настройка ехес-maven-plugin

В корне проекта создан pom.xml. В pom.xml задано:

- координаты проекта (GAV): groupId, artifactId, version;
- свойства компиляции Java;
- ехес-maven-plugin, чтобы запускать mainClass через mvn ехес:java.

Листинг pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>org.example</groupId>
  <artifactId>Test-Maven</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <properties>
    <!-- Компилятор проекта. Можно оставить 17 (или поднять,
если нужно). -->
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</
project.build.sourceEncoding>
  </properties>
  <dependencies>
    <!-- 2) Логирование: SLF4J + Logback -->
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>2.0.13</version>
    </dependency>
    <dependency>
      <groupId>ch.qos.logback</groupId>
      <artifactId>logback-classic</artifactId>
      <version>1.5.6</version>
    </dependency>
```

```

    <!-- 4) JSON: Jackson -->
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.17.2</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <!-- Компиляция -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.13.0</version>
        <configuration>
          <release>${maven.compiler.release}</release>
        </configuration>
      </plugin>
      <!-- Запуск main через mvn exec:java -->
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
          <mainClass>org.example.Main</mainClass>
        </configuration>
      </plugin>
      <!-- 6) SpotBugs -->
      <plugin>
        <groupId>com.github.spotbugs</groupId>
        <artifactId>spotbugs-maven-plugin</artifactId>
        <!-- Важно: свежая версия для совместимости с
        НОВОЙ JDK -->
        <version>4.9.8.2</version>
        <!-- Доп. требование: "сборка падает при high/
        Medium" -->
        <configuration>
          <threshold>Medium</threshold>
          <effort>Max</effort>
          <failOnError>true</failOnError>
        </configuration>
        <executions>
          <execution>
            <id>spotbugs-check</id>
            <phase>verify</phase>
            <goals>

```

```

        <goal>check</goal>
      </goals>
    </execution>
  </executions>
</plugin>
</plugins>
</build>
</project>

```

Сборка и запуск приложения:

MVN CLEAN INSTALL EXEC:JAVA

2) Подключение зависимости для логирования

В pom.xml в <dependencies> добавлены:

- org.slf4j:slf4j-api
- ch.qos.logback:logback-classic

Чтобы логи выглядели предсказуемо, добавлен конфиг Logback.

3) Модификация Main: логирование вместо System.out

Создан класс Main и вывод заменён на логирование.

Листинг src/main/java/org/example/Main.java (полный)

```

package org.example;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Main {

    private static final Logger log =
LoggerFactory.getLogger(Main.class);

    public static void main(String[] args) {

        log.info("App started");

        Person p = new Person("Meow", 19);
    }
}

```

```

        log.debug("Created person: {}", p);
    try {
        String json = JsonUtil.toJson(p);
        log.info("Serialized JSON: {}", json);
        Person restored = JsonUtil.fromJson(json,
Person.class);
        log.info("Deserialized person: {}", restored);
    } catch (Exception e) {
        log.error("Error during JSON processing", e);
    }
    log.info("App finished");
}
}

```

4) Подключение зависимости для JSON

В pom.xml добавлена зависимость com.fasterxml.jackson.core:jackson-databind.

5) Реализация сериализации/десериализации JSON + транзитивные зависимости

Созданы:

- модель данных Person;
- утилита JsonUtil с методами сериализации/десериализации;
- выполнен анализ транзитивных зависимостей через mvn dependency:tree.

Листинг src/main/java/org/example/Person.java

```

package org.example;

public record Person(String name, int age) {}

```

Команда для просмотра транзитивных зависимостей

MVN DEPENDENCY:TREE

[INFO] Building Test-Maven 1.0-SNAPSHOT

[INFO] from pom.xml

[INFO] -----[jar]-----

[INFO]

[INFO] --- dependency:3.7.0:tree (default-cli) @ Test-Maven ---

[INFO] org.example:Test-Maven:jar:1.0-SNAPSHOT

[INFO] +- org.slf4j:slf4j-api:jar:2.0.13:compile

[INFO] +- ch.qos.logback:logback-classic:jar:1.5.6:compile

[INFO] | \- ch.qos.logback:logback-core:jar:1.5.6:compile

[INFO] \- com.fasterxml.jackson.core:jackson-databind:jar:2.17.2:compile

[INFO] +- com.fasterxml.jackson.core:jackson-annotations:jar:2.17.2:compile

[INFO] \- com.fasterxml.jackson.core:jackson-core:jar:2.17.2:compile

[INFO] -----

[INFO] BUILD SUCCESS

[INFO] -----

[INFO] Total time: 0.556 s

[INFO] Finished at: 2026-02-26T00:59:41+03:00

[INFO] -----

В дереве зависимостей видно, что при подключении jackson-databind Maven подтягивает дополнительные артефакты (транзитивно), например jackson-core и jackson-annotations (как типичный кейс).

6) Подключение SpotBugs, запуск проверки и исправление ошибок

SpotBugs подключён в pom.xml как plugin.

MVN SPOTBUGS:CHECK

```
[INFO] >>> spotbugs:4.9.8.2:check (default) > :spotbugs @ Test-Maven >>>
```

```
[INFO]
```

```
[INFO] --- spotbugs:4.9.8.2:spotbugs (spotbugs) @ Test-Maven ---
```

```
[INFO] Fork Value is true
```

```
[INFO] Done SpotBugs Analysis....
```

```
[INFO] Skipping com.github.spotbugs:spotbugs-maven-plugin:4.9.8.2:spotbugs report goal
```

```
[INFO]
```

```
[INFO] <<< spotbugs:4.9.8.2:check (default) < :spotbugs @ Test-Maven <<<
```

```
[INFO]
```

```
[INFO]
```

```
[INFO] --- spotbugs:4.9.8.2:check (default) @ Test-Maven ---
```

```
[INFO] BugInstance size is 0
```

```
[INFO] Error size is 0
```

```
[INFO] No errors/warnings found
```

```
[INFO] -----
```

```
[INFO] BUILD SUCCESS
```

```
[INFO] -----
```

```
[INFO] Total time: 5.112 s
```

```
[INFO] Finished at: 2026-02-26T01:02:50+03:00
```

```
[INFO] -----
```

Дополнительное задание: «сборка падает при ошибках High/Medium»

В методичке: настроить так, чтобы сборка была неуспешной при найденных ошибках определённого уровня.

Это реализовано конфигурацией SpotBugs:

- `<threshold>Medium</threshold>` — считаем дефекты уровня Medium/High достаточными для падения;
- `<failOnError>true</failOnError>` — Maven завершает сборку с ошибкой;
- goal check привязан к фазе verify.

Команда для проверки “как в CI”:

MVN CLEAN VERIFY

Выводы

1. Освоена структура Maven-проекта и роль pom.xml как центрального файла конфигурации сборки и зависимостей.
2. Настроен запуск приложения через exes-maven-plugin и отработана сборка по фазам clean install exes:java.
3. Подключены внешние библиотеки через <dependencies>, подтверждена транзитивность зависимостей.
4. Подключён статический анализатор SpotBugs и реализована политика «сборка падает при ошибках уровня Medium/High».

Ответы на контрольные вопросы

1. Что такое Apache Maven и для чего он используется?

Это инструмент автоматизации сборки на основе модели проекта POM; управляет зависимостями, конфигурацией сборки и жизненным циклом проекта.

2. Как установить Maven на различные ОС?

В Linux пример: `sudo apt install maven` (используя пакетный менеджер). В остальных ОС обычно ставят через пакетные менеджеры/инсталлятор и проверяют командой `mvn`.

3. Какова структура проекта Maven?

Стандартно: `src/main/java` для исходников, `src/main/resources` для ресурсов, артефакты сборки — в `target`. (В методичке описаны директории исходников/ресурсов и `outputDirectory/target/classes`.)

4. Что такое POM файл и какова его роль?

`pom.xml` — XML-файл проекта, Maven первым делом проверяет его корректность; в нём хранятся данные о проекте и сборке.

5. Какова структура файла POM?

Корневой `<project>`, далее заголовок (GAV), `<properties>`, `<dependencies>`, `<build>` (и опционально `<reporting>`).

6. Что такое зависимости и как они определяются?

Это подключаемые библиотеки; объявляются в `<dependencies>`, каждая задаётся тройкой `groupId/artifactId/version` (GAV) и опционально `scope`.

7. Что такое репозиторий Maven и какие виды существуют?

Репозиторий — хранилище артефактов (`jar/pom`). По практике различают локальный репозиторий (на машине) и удалённые (центральный/корпоративный). Локальный репозиторий упоминается в описании фазы `install`.

8. Как добавить зависимость в проект Maven?

Добавить блок `<dependency>...</dependency>` в `<dependencies>` с нужным GAV (и при необходимости `scope`).

9. Что такое плагины в Maven и как используются?

Плагины добавляют действия сборки (проверки качества, запуск, упаковку); подключаются в `<build><plugins>`.

10. Как создать новый Maven-проект командой Maven?

Типовой способ — `archetype (mvn archetype:generate)`. В методичке также показан способ через IDE (“Create Maven project”).

11. Что такое goals и phases, и в чём отличие?

Phase — стадия жизненного цикла (например, `clean`, `install`). Goal — конкретная цель плагина (например, `exec:java`, `spotbugs:check`). Пример в методичке: фазы `clean`, `install` и цель плагина `exec:java`.

12. Как выполнить команду сборки проекта в Maven?

Выполнить `mvn <фазы/цели>`, например `mvn clean install exec:java`.

13. Что такое жизненный цикл сборки?

Это последовательность фаз, через которые Maven проводит проект (очистка → компиляция/тесты/упаковка/установка и т.д.). Пример фаз и их смысл дан для `clean` и `install`.

14. Как настроить profiles для разных сред?

Через `<profiles>` в `pom.xml` (или профили в `settings.xml`) и активацию по условиям/ключу `-P`. Идея: разные наборы настроек под `dev/prod`.

15. Как управлять версиями зависимостей?

Явно указывать `<version>` у зависимости, либо централизовать версии через `<dependencyManagement>` (часто в `parent-pom`).

16. Что такое SNAPSHOT версии и как используются?

-SNAPSHOT означает версию в разработке: артефакт может меняться.

17. Как использовать Maven для отчётов о качестве кода?

Подключить плагины анализа/стилей (`Checkstyle/PMD/SpotBugs` и др.) в `build/reporting` и запускать их goals (пример плагинов показан в методичке).

18. Какие команды Maven используются для очистки, сборки, тестирования и установки?

`mvn clean` (очистка), `mvn test` (тесты), `mvn package` (упаковка), `mvn install` (установка в локальный репозиторий). В методичке подробно разобраны `clean` и `install`.

19. Как интегрировать Maven с Git?

Хранить исходники и `pom.xml` в репозитории, не коммитить `target/`, фиксировать версии, использовать CI (GitHub Actions/GitLab CI) для `mvn clean verify`.

20. Как добавить и настроить сторонний репозиторий?

Через секции `<repositories>` / `<pluginRepositories>` в `pom.xml` либо через `settings.xml` (`mirrors/servers`).

21. Какие scopes зависимостей существуют и для чего?

Основные: `compile` (по умолчанию), `test`, `provided`, `runtime`, `system`, `import`. В методичке упоминается `scope` и пример с `JUnit` для тестов.

22. Чем отличается плагин от зависимости?

Зависимость — библиотека, попадающая в `classpath` приложения. Плагин — инструмент Maven, который выполняет действия в процессе сборки (компиляция, запуск, анализ).

23. Как работает транзитивность зависимостей?

Если подключённая библиотека зависит от других, Maven подтягивает их автоматически (это и требуется проверить в пункте 5 задания).

24. Для чего нужен плагин `surefire`?

Для запуска `unit`-тестов (обычно `JUnit/TestNG`) в фазе `test`.

25. Как исключить транзитивную зависимость?

Через `<exclusions>` внутри `<dependency>` (указать `groupId/artifactId` исключаемого артефакта).